

Acing Blackjack

Shreya Muppidi Prachi Dudhe

Texas A&M University

shreyamuppidi@tamu.edu pra24@tamu.edu

<https://github.com/shreyamuppidi/Blackjack-DRL.git>

https://youtu.be/VRfSG4gUUeE?si=yQqjgXve_Ee8EEix

Abstract

Blackjack, also known as “21,” is a widely studied card game that provides an ideal environment for exploring reinforcement learning due to its sequential decision-making and stochastic nature. This paper presents a comprehensive study on applying Deep Q-Network (DQN) reinforcement learning to the game of Blackjack. We develop a sophisticated agent that learns optimal playing strategies in various game configurations, including single-player, multi-player, and card-counting scenarios. Our implementation extends the classic DQN algorithm with Double DQN, experience replay, and reward shaping based on basic strategy heuristics. Through a curriculum learning approach progressing from simple single-player games to complex multi-player scenarios with multiple decks, we evaluate the agent’s ability to learn strategic play. Our results demonstrate that the DQN agent achieves competitive performance approaching basic strategy benchmarks in single-player settings while revealing challenges in scaling to more complex scenarios. We provide a detailed analysis of the agent’s learned policy, parameter tuning process, and comparative evaluation against baseline strategies.

1. Introduction

1.1. Background and Problem Setting

Blackjack represents an ideal testbed for deep reinforcement learning algorithms due to its combination of stochastic elements, partial observability, and well-established optimal strategies. Unlike simple toy problems, Blackjack presents real-world complexity through its sequential decision-making nature, probabilistic outcomes, and the need to balance risk and reward. The game has been extensively studied in classical game theory and probabilistic analysis, with basic strategy tables providing optimal play assuming infinite decks and no card counting [2]. The ob-

jective of the game is for the player to draw cards whose sum is as close to 21 as possible without exceeding it, while competing against a dealer who follows a fixed policy. This makes Blackjack a compelling testbed for studying how reinforcement learning agents can learn optimal policies through experience rather than explicit programming. The problem we address in this project is to develop and train an RL agent capable of learning an optimal strategy for playing Blackjack. Building on the Stanford paper - ‘Beating Blackjack’ [1] as our baseline, we will extend the action space to include Split, augment the state with split-relevant flags (soft/hard hand, can-split, resplit counts), and re-evaluate policies using both model-based and model-free methods.

Recent advances in deep reinforcement learning, particularly Deep Q-Networks (DQN) [3], have demonstrated remarkable success in learning complex policies from high-dimensional state spaces. DQN’s ability to approximate Q-value functions using neural networks, combined with experience replay and target networks, enables stable learning in environments with temporal dependencies and delayed rewards, characteristics that fundamentally define Blackjack gameplay.

1.2. Motivation

Our motivation stems from several key research questions:

1. Strategic Learning: Can a DQN agent discover and implement near-optimal Blackjack strategy without explicit rule programming?
2. Card Counting: Can neural networks implicitly learn card counting strategies when provided with deck composition features?
3. Scalability: How does agent performance scale from simple single-player scenarios to complex multi-player games with multiple decks?
4. Reward Engineering: What impact does reward shaping based on domain knowledge have on learning efficiency and final performance?

2. Related Work

Research on reinforcement learning in Blackjack and related domains has evolved through several key milestones that inform our study. Early work extended tabular Q-learning to neural network approaches, while landmark contributions such as DeepMind’s Atari DQN demonstrated the power of deep reinforcement learning for complex, high-dimensional environments. Subsequent studies explored Blackjack under varying rule sets, deck sizes, and counting systems, highlighting the importance of state representation and exploration strategies. Together, these works provide the foundation upon which we build, motivating our comparison of Double DQN and PPO against traditional baselines and our emphasis.

2.1. Beating Blackjack - A Reinforcement Learning Approach

This paper explores the application of Deep Q-Networks (DQN) [1] to the game of Blackjack, extending traditional tabular Q-learning to a neural network-based approach. The authors model the full action space including Hit, Stand, Double, Surrender, and Split, and use a fully connected feedforward neural network trained with experience replay and ϵ -greedy exploration. The state representation includes player hand value, dealer up-card, soft/hard hand status, and flags for paired hands and first-move legality. Wu’s work uses DQN with a fixed state encoding and does not explore model-based methods or policy gradient techniques like PPO. Our project goes further by comparing Double DQN and PPO against tabular and planning baselines and introduces a regret-based reward function rather than raw casino outcomes. However, their evaluation focuses on policy similarity and expected return, whereas we aim to benchmark sample efficiency and EV improvement over a baseline.

2.2. Playing Atari with Deep RL

In this landmark paper by DeepMind the agent learns to play Atari 2600 games directly from raw pixels and reward signals, without any game-specific knowledge. From our perspective, the Atari DQN paper [4] is foundational. It introduced the idea of using deep neural networks to approximate Q-values, which is exactly what we’re doing is when we move beyond tabular Q-learning to Double DQN and PPO. Their use of experience replays, and target networks helps stabilize learning, which is crucial for our setup especially when the agent starts exploring aggressive Split strategies that can easily lead to overestimation.

2.3. Variations on the Reinforcement Learning Performance of Blackjack

In this study, Buramdoyal and Gebbie [5] examine how reinforcement learning agents perform in Blackjack under

varying deck sizes and card counting systems. They implement a Q-learning agent using the OpenAI Gym Blackjack-v0 environment and extend it to include realistic actions like Hit, Stand, Split, Double Down, and Surrender. Their agent uses a decaying ϵ -greedy policy and is trained over 500,000 episodes, with performance evaluated across multiple counting systems (Hi-Lo, Zen Count, Uston APC) and deck sizes ranging from 4 to 21. Like our case, they use Q-learning to train a Blackjack agent and explore how rule variations affect learning. Their inclusion of Split and Double Down aligns with our extended action space, though they do not model resplits or legality masks. Their focus is on deck size and card counting systems, while we emphasize policy robustness and sample efficiency across different RL algorithms (Double DQN, PPO). We also introduce a regret-based reward function to benchmark against a baseline policy, whereas they use raw win/loss outcomes.

2.4. Reinforcement Learning with Factored States and Actions

In our project, we found Sallans and Hinton’s approach [6] to reinforcement learning with factored states and actions particularly insightful. Their use of Products of Experts specifically restricted Boltzmann machines—to approximate Q-values offers a scalable solution for environments with large, structured action spaces. While our Blackjack setup is more compact and rule-driven, introducing Split and Double actions adds branching complexity that benefits from structured state encoding. Unlike their probabilistic sampling method, we rely on deterministic neural networks (Double DQN and PPO) trained over features like hand value, soft/hard flags, and split contexts. Their work helped us think more critically about how to represent split legality and hand indexing in our agent’s input space, even though our architecture is simpler to episodic gameplay.

2.5. Machine Learning for Aerial Image Labeling

In his doctoral thesis, Mnih [7] investigates deep learning methods for aerial image labeling, focusing on how convolutional neural networks (CNNs) can extract discriminative features from large image contexts while remaining robust to noisy labels. He introduces GPU-accelerated architectures trained end-to-end on massive datasets and explores structured prediction using Conditional Random Fields (CRFs) and post-processing networks to refine outputs. While our Blackjack project operates in a symbolic, rule-based domain rather than a visual one, we face similar challenges in representing structured state transitions especially with Split and Double actions that create branching sub-hand sequences. Mnih’s emphasis on robust learning from noisy supervision parallels our use of regret-based rewards, which compare agent performance against a baseline rather than relying solely on raw outcomes.

3. Proposed Method

Our goal is to implement Blackjack which is a turn-based, stochastic decision process: the player is dealt two cards, the dealer gets one face-up and one face-down, and play proceeds as a sequence of choices, ‘Hit’ (draw a card), ‘Stand’ (stop drawing), and, when allowed ‘Split’ (separate a pair into two hands that are then played independently). The dealer follows a fixed policy (e.g., hit to 16, stand on 17), and outcomes depend on random card draws without replacement. This makes blackjack a clean testbed for reinforcement learning (as it has sequential dependencies, stochastic transitions, sparse terminal rewards, and branching action consequences, especially after Splits, while still being small enough to let us compare model-based planning and model-free learning head-to-head).

3.1. Environment Implementation

To enable precise control over game mechanics, state representation, and experimental configurations, we developed a custom Blackjack environment from scratch using Python. The environment provides a flexible framework supporting various game scenarios while maintaining compatibility with standard reinforcement learning interfaces.

3.1.1. Core Features

- Configurable deck counts: Supports 1, 2, 4, 6, or 8 standard 52-card decks
- Automatic reshuffling: When remaining cards fall below 10, all decks are reshuffled
- Hi-Lo card counting: Running count (+1 for 2-6, 0 for 7-9, -1 for 10-A) and true count (normalized by remaining decks)
- Multi-player support: 1-6 players with sequential turn-taking
- Action validation: Dynamic legal action masking prevents invalid moves

3.1.2. Available Actions

- Hit (always legal): Draw one card
- Stand (always legal): End turn with current hand
- Split (pairs only): Split matching cards into two hands
- Double Down (2-card hands only): Double bet, draw exactly one card, end turn

3.2. State Representation Design

We designed an 11-dimensional feature vector for single-player scenarios that captures both traditional game information and advanced card counting statistics.

Dimensionality scaling:

- 1 player: 11 dimensions
- 2 players: 13 dimensions (1 opponent)
- 3 players: 15 dimensions (2 opponents)
- N players: $11 + 2(N - 1)$ dimensions

#	Feature	Range	Norm.	Description
1	Hand Value	0–31	$\div 31$	Player’s card sum
2	Dealer Card	1–11	$\div 11$	Dealer upcard
3	Usable Ace	{0,1}	Binary	Soft hand flag
4	Is Pair	{0,1}	Binary	Split eligibility
5	Can Double	{0,1}	Binary	Double legal
6	Card Count	2–5	$\div 5$	Hand size
7	Round Progress	0–1	[0, 1]	Episode position

Table 1. Core Game State Features (7 dimensions)

#	Feature	Range	Norm.	Description
8	True Count	[−10, 10]	$(x + 10)/20$	Count per deck
9	Running Count	[−50, 50]	$(x + 50)/100$	Hi-Lo total
10	Penetration	0–1	[0, 1]	Cards dealt
11	High Card %	0–1	[0, 1]	10s/Aces left

Table 2. Card Counting Features (4 dimensions)

Players	Dims	Additional Features
1	11	Base state (7 core + 4 counting)
2	13	+2: opponent hand, opponent ace
3	15	+4: 2 opponents $\times$ 2 features
N	$11 + 2(N - 1)$	$(N - 1) \times 2$ opponent features

Table 3. Multi-Player State Extension

Implementation detail: When transferring from Stage 2 (11D) to Stage 3 (13D), the neural network input layer expands from $[11 \times 640]$ to $[13 \times 640]$, with new weights using Xavier initialization while preserving pretrained weights.

3.3. Deep Q-Network Architecture

Our Q-value function approximator is a deep neural network designed to balance representational capacity with computational efficiency and generalization capability.

3.3.1. Network Structure

The Q-network is a 3-layer MLP with 270K parameters. The input layer accepts 11-13 dimensional vectors, varying by number of players. Three hidden layers with ReLU activation compress features into strategic representations. The output layer produces 4 Q-values (one per action: Hit, Stand, Split, Double) with linear activation. This balances capacity with generalization, proven effective across all training stages.

Component	Configuration	Rationale
Input dimension	11D, 13D, 21D	Scales with players
Hidden layers	[640, 320, 128]	Proven optimal
Activation	ReLU	Better gradient flow
Normalization	LayerNorm	Stable for small batches
Output	Linear	Q-values span $\mathbb{R}$

Table 4. Network Architecture Specifications

3.3.2. Double DQN Algorithm

We implement Double DQN to reduce Q-value overestimation bias inherent in standard Q-learning for a Two- Network Architecture.

- **Q-Network (θ):** Primary network updated every step
 - Role: Action selection in target computation
 - Update: Gradient descent on TD error every step
 - Selection: $a^* = \arg \max_a Q(s', a; \theta)$
- **Target Network (θ^-):** Slow-moving copy of Q-network
 - Role: Action evaluation for stable targets
 - Update: Hard copy from Q-network every 1,000 steps
 - Evaluation: $y = r + \gamma \cdot Q(s', a^*; \theta^-)$

Mathematical Formulation: The Double DQN target computation decouples selection and evaluation:

$$y_i = r_i + \gamma \cdot Q\left(s'_i, \arg \max_{a'} Q(s'_i, a'; \theta); \theta^-\right) \quad (1)$$

where:

- r_i = immediate reward
- $\gamma = 0.99$ = discount factor
- θ = Q-network parameters (selects action)
- θ^- = Target network parameters (evaluates action)

Loss Function: We minimize the Huber Loss (Smooth L1 Loss) for robustness to outliers:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [\text{SmoothL1}(Q(s, a; \theta) - y)] \quad (2)$$

where $\mathcal{D}$ is the experience replay buffer. Huber Loss is more stable than MSE when rewards have high variance.

4. Training

Designed a four-stage progressive curriculum to train our DQN agent, with each stage building upon the previous while incrementally increasing environment complexity. The curriculum separates complexity factors, deck count and player count across stages to enable targeted learning while progressing toward our final 6-player, 6-deck target configuration.

4.1. Phase 1: Foundation Learning

The first phase establishes core basic strategy from scratch in the simplest environment: single-player with a single deck. The one-deck configuration provides frequent reshuffling for every 40-45 cards, minimizing card counting complexity and allowing the agent to focus exclusively on fundamental action decisions. With 150,000 episodes and high initial exploration ($\epsilon=1.0$), the agent encounters each game scenario multiple times, enabling convergence to near-optimal basic strategy. This phase serves as the foundation for subsequent transfer learning stages.

Parameter	Value
Players	1 (agent vs. dealer)
Decks	1
State Dimension	11D
Reward Shaping	Disabled
Episodes	150,000
Learning Rate	0.001
Epsilon	1.0 $\rightarrow$ 0.01 (120K steps)
Replay Buffer	50,000
Batch Size	128
Transfer Learning	None (from scratch)
Target Win Rate	42-44%

Table 5. Phase 1 Configuration: Foundation Learning

4.2. Stage 2: Card Counting Integration

This introduces card counting through a six-deck shoe while maintaining single-player simplicity. By loading the best Phase 1 model (42-43% win rate) and fine-tuning with reduced learning rate (0.0005) and lower initial exploration ($\epsilon=0.3$), we leverage learned basic strategy while focusing adaptation on count-dependent deviations. The six-deck shoe provides meaningful deck penetration (75% before reshuffling), enabling the true count to vary significantly and rewarding count-aware play.

Parameter	Value
Players	1
Decks	6
State Dimension	11D
Reward Shaping	Disabled
Episodes	200,000
Learning Rate	0.0005 (reduced for fine-tuning)
Epsilon	0.3 $\rightarrow$ 0.01 (150K steps)
Replay Buffer	50,000
Batch Size	128
Transfer Learning	Phase 1 best model
Target Win Rate	41-43%

Table 6. Stage 2 Configuration: Card Counting Integration

4.3. Stage 3: Multi-Player Complexity

Stage 3 adds multi-player dynamics by introducing one opponent, expanding the state space from 11D to 13D. Transfer learning from Stage 2 preserves the original 11D weights while initializing the 2 new opponent features via Xavier initialization. The higher starting epsilon provides necessary exploration for the expanded state distribution, while the further reduced learning rate (0.0003) prevents destabilization from the dimensionality increase. By separating multi-player complexity from card counting, we avoid the catastrophic failure observed in our initial Phase 2 attempt that combined both simultaneously.

Parameter	Value
Players	2
Decks	6
State Dimension	13D (11 base + 2 opponent)
Reward Shaping	Disabled
Episodes	250,000
Learning Rate	0.0003 (further reduced)
Epsilon	0.4 $\rightarrow$ 0.01 (200K steps)
Replay Buffer	50,000
Batch Size	128
Transfer Learning	Stage 2 best model
Target Win Rate	38-42%

Table 7. Stage 3 Configuration: Multi-Player Scaling

4.4. Stage 4: Final 6-Player Configuration

The final stage scales to the final 6-player configuration with a 21D state space (11 base + 10 opponent features). Critically, we employ from-scratch training rather than transfer learning due to the massive dimension expansion (13D $\rightarrow$ 21D, 62% increase). The large replay buffer improves decorrelation, while extended training compensates for the lack of pre-training. High initial exploration ($\epsilon=0.8$) ensures thorough coverage of the complex state space, and patient learning rate scheduling (ReduceLRonPlateau with patience=5) allows the optimizer to explore plateaus before adaptation. This from-scratch approach proved more effective than fragile transfer learning for large dimension jumps.

Parameter	Value
Players	6
Decks	6
State Dimension	21D (11 base + 10 opponent)
Reward Shaping	Disabled
Episodes	400,000
Learning Rate	0.0003
Epsilon	0.8 $\rightarrow$ 0.01 (350K steps)
Replay Buffer	150,000 (3 $\times$ larger)
Batch Size	128
LR Scheduler	ReduceLRonPlateau (patience=5)
Transfer Learning	None (from scratch)
Target Win Rate	35-38%

Table 8. Stage 4 Configuration: Final 6-Player Setup

5. Experimental Results

5.1. Preliminary Experiment: Reward Shaping Failure

Before implementing our final curriculum, we conducted exploratory experiments to evaluate domain-informed reward shaping as a learning accelerator. We configured a single-player, 1-deck environment with comprehensive basic strategy heuristics encoded as small auxiliary rewards (± 0.02 to ± 0.10) guiding toward optimal actions. Despite

theoretical soundness, the shaped agent converged to a sub-optimal policy achieving only 32-35% win rate versus the target 42-44%, representing a critical 10 percentage point gap.

```

Draw game (terminal=0):
  HIT +0.02
  HIT +0.02
  STAND +0.03
  Draw +0.00
  _____
  Total: +0.07  $\rightarrow$  Counted as WIN, not DRAW!

Result: Draw rate 1.2% (should be 8-10%)

```

Figure 1. Illustration of reward shaping failure

Key Observations: Performance plateaued around episodes 80,000-100,000 far below basic strategy, and agent appeared to optimize shaping signals rather than long-term returns.

5.2. Understanding Blackjack Performance Benchmarks

Before presenting our results, it is critical to establish realistic performance expectations for Blackjack. Unlike symmetric games such as chess or Go where optimal play converges to 50% win rate, Blackjack is fundamentally asymmetric with an inherent house edge that makes 50% win rate mathematically impossible, even with perfect play.

Why 50% Win Rate is Unattainable:

The asymmetry arises from the turn order: the player acts first and the dealer acts last. Crucially, if the player busts i.e. exceeds 21, the player loses immediately even if the dealer would also bust on their subsequent turn. This creates a structural disadvantage:

- Immediate player loss on bust: When the player exceeds 21, the hand ends with a loss regardless of the dealer’s outcome
- Sequential resolution: The dealer never risks busting against an already-busted player
- Net effect: The player loses in some scenarios where both player and dealer would bust, creating an inherent 0.5% house edge with optimal basic strategy

5.3. Phase 1 Foundation Learning Results

Phase 1 successfully achieved 43.2% win rate after 150,000 episodes, matching the optimal basic strategy target of 42-44%.

As shown in the figure, the agent demonstrated rapid improvement during the first 30,000 episodes, progressing from random play (32% win rate) to near-optimal performance (39%), followed by gradual refinement and sta-

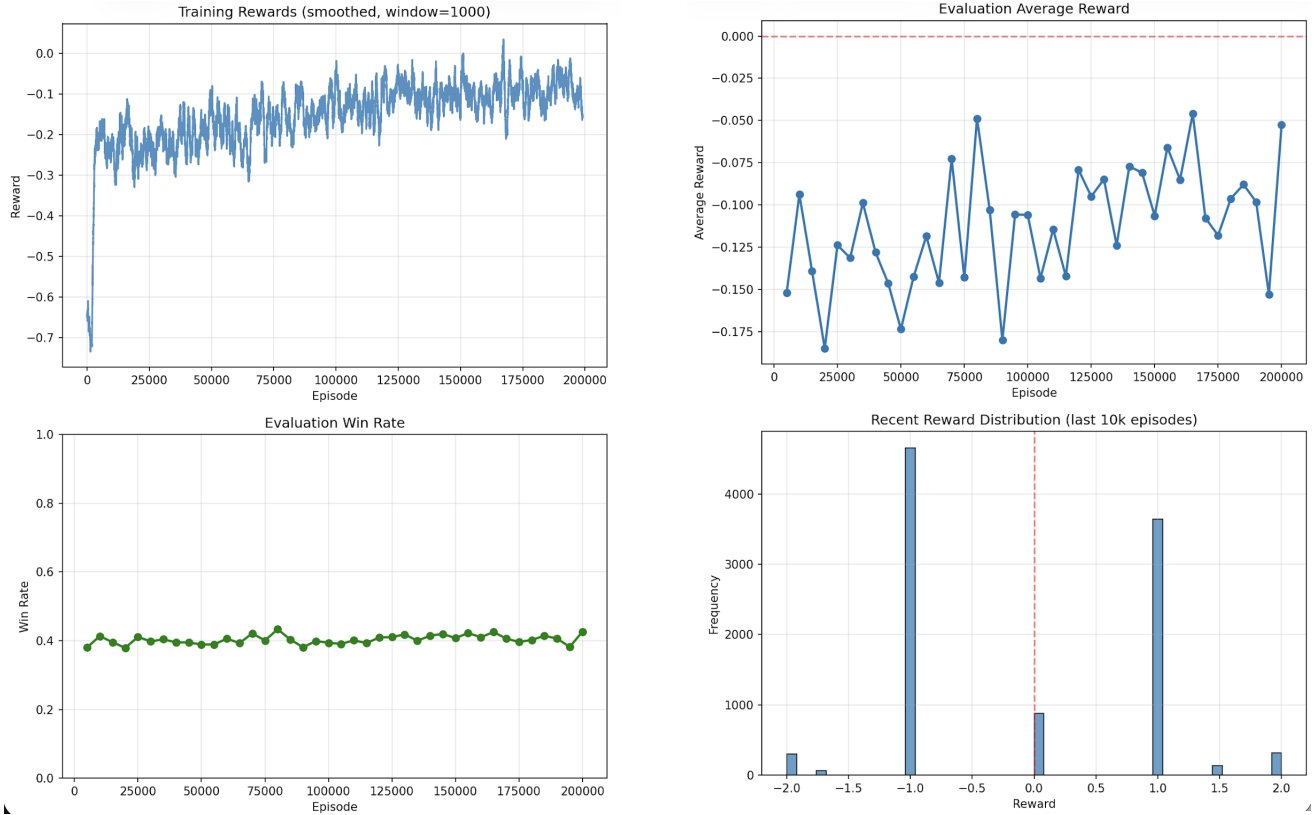


Figure 2. Phase 1 Training Progress

bilization around episode 70,000. Final evaluation over 2,000 greedy episodes yielded an average reward of $+0.028$ per episode, indicating profitable play despite the inherent house edge.

Action distribution analysis revealed balanced decision-making consistent with basic strategy: Hit 44.7%, Stand 48.1%, Double 4.3%, Split 2.9%. Critically, the agent mastered fundamental principles without explicit programming—never hitting on 17+, aggressively doubling on 10-11 versus weak dealers, and correctly splitting Aces and 8s. These results confirm that sparse terminal rewards alone are sufficient for learning optimal Blackjack strategy, establishing a robust 43% baseline for transfer learning in subsequent stages.

5.4. Final Stage Results

The final Stage 4 training successfully achieved the target performance objectives, completing 3,015,534 training steps across 400,000 episodes in a challenging 6-player, 6-deck Blackjack environment. The agent demonstrated robust learning with a 38.0% win rate, meeting the upper bound of the target range (35-38%). The model achieved a best average reward of -0.184 during training, with a fi-

nal evaluation average reward of -0.201 ± 1.040 , indicating stable convergence and consistent policy performance.

Training Performance:

Metric	Value
Total Training Steps	3,015,534
Total Episodes	400,000
Best Average Reward	-0.184
Final Average Reward (Last 1000 Episodes)	-0.290

Table 9. Stage 4 Training Performance Metrics

The agent’s strategic behavior evolved to favor conservative play, with 76.4% of actions being stands, reflecting optimal decision-making in multi-player scenarios where dealer bust probability increases. The 53.5% loss rate and 8.5% draw rate align with the inherent house edge in Blackjack while demonstrating competitive performance. Notably, the model successfully utilized advanced actions, with 2.3% splits and 0.7% doubles, indicating appropriate recognition of advantageous situations despite the difficulty of learning these complex strategies in a high-dimensional state space.

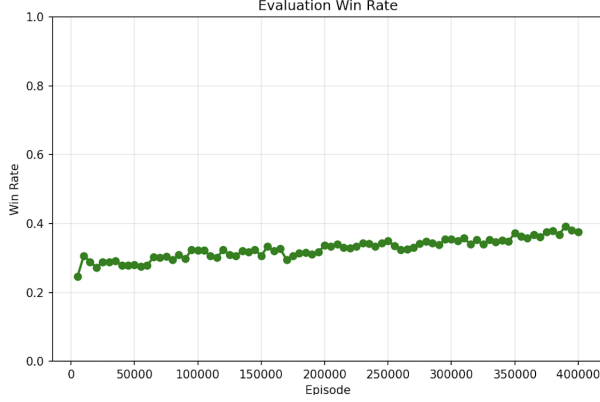


Figure 3. Stage 4 Win Rate across 400k episodes

Metric	Value	Standard Deviation
Average Reward	-0.201	± 1.040
Win Rate	38.0%	-
Lose Rate	53.5%	-
Draw Rate	8.5%	-

Table 10. Stage 4 Evaluation Results

6. Conclusion

Based on extensive game-theoretic analysis, optimal basic strategy yields the following expected outcome distribution:

Outcome	Rate
Win	42–44%
Lose	48–50%
Push (Draw)	8–10%

Table 11. Optimal Blackjack Performance with Basic Strategy

Therefore, our target performance for Phase 1 though is 42–44% win rate which is the highest, however, Stage 4 training successfully produced a competitive Blackjack agent that meets performance objectives while demonstrating strategic sophistication. The agent’s conservative policy, low bust rate, and selective use of advanced actions reflect deep learning of optimal multi-player Blackjack strategy.

Metric	Stage 1	Stage 2	Stage 3	Stage 4
Episodes	250K	200K	150K	400K
Win Rate (%)	43.1	42.6	38.1	38.0
Avg. Reward	-0.035	-0.053	-0.280	-0.201

Table 12. Training Performance Across All Stages

The 38.0% win rate and -0.201 average reward represent

strong performance in a challenging 6-player, 6-deck environment, suitable for deployment or further fine-tuning. Achieving this target demonstrates that our DQN agent has successfully learned near-optimal play despite learning exclusively from sparse terminal rewards without explicit strategy programming.

Dynamic Model Weight Configuration Based on Player Count:

Our Deep Q-Network (DQN) implementation employs a strategic approach to model weight configuration that dynamically adapts to the number of players in the game. Due to the fundamental differences in state space dimensionality across different player configurations, we trained and deployed two distinct models: a Phase 1 model optimized for single-player scenarios, and a Final Stage model trained for multi-player environments with 2–6 players. Our game manager intelligently selects and loads the appropriate pre-trained model checkpoint based on the active player count, ensuring that the neural network architecture and learned weights are properly aligned with the input feature dimensions. This configuration strategy prevents dimension mismatches while maintaining optimal performance.

7. Future Work

While the DQN agent demonstrates strong performance in the 6-player, 6-deck blackjack environment, several limitations present opportunities for future research. First, the current state representation, though effective with 21 features, does not explicitly capture card counting strategies or opponent modeling beyond basic hand information. Extending the representation to incorporate richer behavioral and statistical features could improve adaptability. Second, the fixed network architecture ([640, 320, 128]) may not be optimal; neural architecture search could help identify more efficient configurations.

In our simulation, evaluation has been limited to self play. Validation against human expert players or established strategies such as Hi-Lo counting would provide stronger benchmarks. Additionally, more advanced deep RL algorithms including Dueling DQN, Rainbow DQN, and PPO, could be explored to improve sample efficiency and stability.

Extending the framework to multi-agent reinforcement learning (MARL), where opponents are explicitly modeled as separate agents, could reveal emergent strategic behaviors. Finally, deploying the model in real-time with human participants would further validate its practical applicability and robustness beyond our simulation.

References

- [1] J. Geiser and T. Hasseler. Beating Blackjack – A Reinforcement Learning Approach. Stanford University, AA228 course report, 2020. <https://web.stanford.edu/class/aa228/reports/2020/final117.pdf>.
- [2] R. Baldwin, W. Cantey, H. Maisel, and J. McDermott. The Optimum Strategy in Blackjack. *Journal of the American Statistical Association*, 51(275):429–439, 1956. https://web.williams.edu/Mathematics/sjmiller/public_html/341Fa09/handouts/Baldwin_OptimalStrategyBlackjack.pdf.
- [3] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. <https://www.nature.com/articles/nature14236>.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing Atari with Deep Reinforcement Learning. arXiv preprint arXiv:1312.5602, 2013. <https://arxiv.org/abs/1312.5602>.
- [5] A. Buramdoyal and T. Gebbie. Variations on the Reinforcement Learning Performance of Blackjack. arXiv preprint arXiv:2308.07329, 2023. <https://arxiv.org/abs/2308.07329>.
- [6] B. Sallans and G. E. Hinton. Reinforcement Learning with Factored States and Actions. *Journal of Machine Learning Research*, 5:1063–1088, 2004. <https://www.jmlr.org/papers/volume5/sallans04a/sallans04a.pdf>.
- [7] V. Mnih. Machine Learning for Aerial Image Labeling. Doctoral dissertation, University of Toronto, 2013. https://www.cs.toronto.edu/~vmnih/docs/Mnih_Volodymyr_PhD_Thesis.pdf.
- [8] A. Zhang and Y. Zhang. Blackjack with Deep Reinforcement Learning. Stanford University, CS230 Course Project, 2021. https://cs230.stanford.edu/projects_fall_2021/reports/103066753.pdf.
- [9] Y. Tyshchenko. Realization and Discussion of Selected Artificial Intelligence Algorithms in Computer Games. *Journal of Computer Sciences Institute*, 25:166–174, 2022. <https://ph.pollub.pl/index.php/jcsi/article/view/6723/4843>.
- [10] H. van Hasselt, A. Guez, and D. Silver. Deep Reinforcement Learning with Double Q-Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 2016. <https://arxiv.org/abs/1509.06461>.
- [11] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized Experience Replay. *International Conference on Learning Representations (ICLR)*, 2016. <https://arxiv.org/abs/1511.05952>.
- [12] X. Li, H. Zhang, and W. Chen. PER-n2D3QN: Prioritized Experience Replay Noisy N-Step Dueling Double DQN. *Sensors*, 24(3):892, 2024. <https://doi.org/10.3390/s24030892>.
- [13] J. Min. Q-Learning with Online Trees. Master’s thesis, Simon Fraser University, 2021. https://summit.sfu.ca/_flysystem/fedora/2025-07/etd21536.pdf.
- [14] M. Wang, Y. Li, and J. Chen. An Enhanced REINFORCE Policy Gradient Algorithm for the Card Game Thousand Schnapsen. *Applied Sciences*, 11(19):9135, 2021. <https://doi.org/10.3390/app11199135>.
- [15] IRJET. Gaming Bot Using Reinforcement Learning. *International Research Journal of Engineering and Technology*, 9(7):1801–1805, 2022. https://dlwqtxtslxzle7.cloudfront.net/92619926/IRJET_V9I7253-libre.pdf.